

Building the Composite and Position Resources



Day 3 (PM) - Session 6

Three Layers - Do Them in Order



Layer 1 - Fix all atomic routes [~25 min]

All routes return snake_case shapes, no envelope wrappers, res.json() on every route.

Layer 2 - Upgrade the vehicle composite [~60 min]

GET /vehicles/:vehicleId returns vehicle fields plus a nested last_ping object (most recent ping by timestamp, or null).

Layer 3 - Add /last-position [~35 min]

GET /vehicles/:vehicleId/last-position returns position data only - no vehicle metadata.

Do not begin Layer 2 until all Layer 1 routes pass a REST-client check. Starting Layer 2 with uncorrected shapes produces an inconsistent API that fails the Day 3 checkpoint even if the composite is correct.

S5 Authoritative Shapes - Your Reference

ATOMIC RESOURCES

Province

province_id | name

District

district_id | name | province_id *

PoliceStation

station_id | name | district_id *

Vehicle (atomic)

vehicle_id | reg_number | device_id | station_id *

LocationPing (GET /vehicles/:id/pings)

ping_id | vehicle_id | timestamp | lat | lng | speed

** = FK field - generators often omit these*

COMPOSITE + DERIVED

Vehicle Composite GET /vehicles/:vehicleId

vehicle_id | reg_number | device_id | station_id
last_ping: {
 ping_id | vehicle_id | timestamp
 lat | lng | speed
} or null (if no pings exist)

/last-position GET /vehicles/:vehicleId/last-position

vehicle_id | timestamp | lat | lng | speed

Collections (all resource types)

Arrays of the corresponding atomic shape. No pagination envelope until S9.

Layer 1 - Atomic Shapes

Layer 1 generate prompt:

"Update these Express GET routes to return JSON matching the shapes exactly. Use res.json().
No envelope wrappers. Data is in seed.json.

GET /provinces -> [{ province_id, name }]

GET /provinces/:id -> { province_id, name }

GET /districts -> [{ district_id, name, province_id }]

GET /districts/:id -> { district_id, name, province_id }

GET /stations -> [{ station_id, name, district_id }]

GET /stations/:id -> { station_id, name, district_id }

GET /vehicles -> [{ vehicle_id, reg_number, device_id, station_id }]

GET /vehicles/:id/pings -> [{ ping_id, vehicle_id, timestamp, lat, lng, speed }]"

9 routes from S4 - fixing 8 today

/provinces

/provinces/:id

/districts

/districts/:id

/stations

/stations/:id

/vehicles

/vehicles/:id/pings

Layer 2 upgrades:

/vehicles/:id (composite)

Layer 3 adds:

/vehicles/:id/last-position

Layer 1 Critique - Four Generator Mistakes



1 camelCase field names

vehicleId, regNumber, stationId -> must be vehicle_id, reg_number, station_id (snake_case throughout)

2 Envelope wrappers

{ "data": [...], "status": "success" } -> remove it. Return the flat array or object directly.

3 Missing FK fields

province_id on District, district_id on Station, station_id on Vehicle are often dropped. These must be present - they are how police traverse the hierarchy.

4 res.send() instead of res.json()

res.send(obj) does not guarantee application/json. Use res.json(obj) on every route.

Before moving to Layer 2: open your REST client and call each route. Check field names, confirm no wrapper object is present, and inspect the Content-Type header in the response.

Layer 2 - The Composite

Layer 2 generate prompt:

"Update GET /vehicles/:vehicleId to return a vehicle composite. Include all vehicle fields plus a last_ping field - the most recent LocationPing.

Find last_ping: filter pings where vehicle_id matches, sort by timestamp descending, take [0].
Return null if no pings exist for this vehicle.

Response: { vehicle_id, reg_number, device_id, station_id, last_ping: { ping_id, vehicle_id, timestamp, lat, lng, speed } | null }

Do not embed the full pings array. One ping only."

Target shape:

GET /vehicles/:vehicleId

```
{  
  vehicle_id, reg_number,  
  device_id, station_id,  
  last_ping: {  
    ping_id, vehicle_id,  
    timestamp, lat, lng, speed  
  } or null  
}
```

Before testing - ask yourself:

How does your code find the MOST RECENT ping?

Is last_ping a nested object or flat fields?

Layer 2 Critique - Three Generator Mistakes

1 Wrong sort direction (or no sort at all)

Generators sort ascending or rely on seed.json order. Code must sort by timestamp descending before [0]. Ask before testing: 'How does your code know which ping is most recent?'

2 Flat fields instead of a nested last_ping object

{ last_lat, last_lng, last_timestamp } on the vehicle object is wrong. The shape requires last_ping: { ping_id, vehicle_id, timestamp, lat, lng, speed }.

3 Embedding the full pings array in the composite

last_ping holds exactly ONE ping. The full history belongs to the separate /pings collection route.

This flat-fields mistake has appeared three times in this module: in your S1 ER model, in the S5 shape critique, and now in generated code. Generators make this error consistently. Catching and correcting it is what scores under API Design.

Layer 3 - Last Position

Layer 3 generate prompt:

"Add GET /vehicles/:vehicleId/last-position to Express. Returns most recent position only - no vehicle metadata.

Filter pings where vehicle_id matches, sort by timestamp descending, take [0]. Return 404 if no pings exist for this vehicle.

Route path must be: /last-position
(lowercase hyphen, noun, no verb)

Response: { vehicle_id, timestamp, lat, lng, speed }"

Target shape:

GET /vehicles/:vehicleId/last-position

```
{  
  vehicle_id,  
  timestamp,  
  lat,  
  lng,  
  speed  
}
```

This is NOT the composite.

No vehicle metadata (reg_number, device_id).

No last_ping wrapper.

Layer 3 Critique - Naming and the Helper



1 Verb in the route path

/getLastPosition, /vehicles/:id/calculatePosition -> must be /last-position (noun, lowercase, hyphenated). The same naming rule from S3 applies here. Spot it in generated code before running the route.

2 Returning the full vehicle composite from /last-position

/last-position is position-only: vehicle_id, timestamp, lat, lng, speed. No reg_number, no device_id, no last_ping wrapper. These belong to the composite route, not here.

Bonus critique (not a checkpoint requirement): if your Layer 2 composite and your Layer 3 route share identical sort-filter-slice logic, extract a getLastPing(vehicleId) helper. This is a sign you read your generated code rather than just deployed it. Micro-viva question 4 will ask about this.

Fallback - Consolidated Prompt

Use only when generated code fails to compile or produces wrong shapes across multiple layers - not as a Layer 1 shortcut.

```
"Rewrite the Express routes file. Data from seed.json:
const data = JSON.parse(fs.readFileSync('./seed.json','utf8'));
```

Helper - get the most recent ping for a vehicle:

```
function getLastPing(vehicleId) {
  const pings = data.pings.filter(p => p.vehicle_id === vehicleId)
    .sort((a,b) => new Date(b.timestamp)-new Date(a.timestamp));
  return pings[0] || null; }
```

Routes (all use res.json(), no envelope wrappers):

```
GET /provinces -> res.json(data.provinces)
GET /provinces/:id -> res.json(data.provinces.find(p => p.province_id === req.params.id))
(repeat pattern for /districts and /stations)
GET /vehicles -> res.json(data.vehicles)
GET /vehicles/:id -> const v = data.vehicles.find(v => v.vehicle_id === req.params.id);
  res.json({ ...v, last_ping: getLastPing(req.params.id) })
GET /vehicles/:id/pings -> res.json(data.pings.filter(p => p.vehicle_id === req.params.id))
GET /vehicles/:id/last-position -> res.json(getLastPing(req.params.id))"
```

Day 3 Checkpoint - Binary Pass Criteria



- [1] All atomic routes return S5-authoritative snake_case shapes with no envelope wrappers and correct FK fields present.
- [2] GET /vehicles/:vehicleId returns the composite shape: vehicle fields plus last_ping as a nested object (the most recent ping by timestamp, or null).
- [3] GET /vehicles/:vehicleId/pings returns an array of LocationPing shapes (route confirmed correct).
- [4] GET /vehicles/:vehicleId/last-position returns the position-only shape. Route path is /last-position (no verb, hyphenated).
- [5] All responses have Content-Type: application/json (verify in your REST client response headers).
- [6] Changes committed to repo. Public URL updated. Instructor can verify all eleven routes.

Assessment dimension: API Design - §6 (representation), §4.3 (composite), §4.5 (processing function resource).

Micro-Viva Questions - Day 3



Q1 Show me GET /vehicles/{one of your vehicle IDs}. Is last_ping a nested object or flat fields? What would be wrong with returning last_lat and last_lng instead?

Q2 Show me GET /vehicles/{same ID}/last-position. What is different about this response compared to the composite? When would a client choose one over the other?

Q3 Your code finds the most recent ping for the composite. How does it know which ping is most recent? Show me that logic in the code.

Q4 Does your /last-position route contain the same logic as your composite route? If yes, is there a way to avoid duplicating it?

Q5 Check the response headers for any of your routes. What Content-Type is your API returning, and what in your code produces it?

5 minutes per student. Record any generated code a student cannot explain - this maps directly to the module viva eligibility gate.

Questions & Discussion



*NB6007CEM - Web API Development | S6
CompositeAndPosition*

Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 21 Jun 2026